

Cadastro e controle de produtos

Aluno: Carlos Eduardo Silva Ferreira

Matrícula: 202408542283

Professor: Alessandro Calin

Testes do menu e arquivo criado

Registro de execução - 01_cadastro_mouse

```
PS C:\programacao_software_basico_c\trabalho> .\produtos.exe
===== CONTROLE DE PRODUTOS =====
1. Cadastrar produto
2. Listar produtos
3. Buscar produto
4. Calcular valor do estoque
5. Atualizar quantidade
6. Sair
Escolha uma opcao: 1
=== Cadastro de produto ===
Codigo: 1001
Nome: Mouse sem fio
Categoria: Perifericos
Quantidade em estoque: 2
Preco unitario: R$ 149.90
Produto cadastrado com sucesso.
===== CONTROLE DE PRODUTOS =====
1. Cadastrar produto
2. Listar produtos
3. Buscar produto
4. Calcular valor do estoque
5. Atualizar quantidade
6. Sair
Escolha uma opcao: 6
Programa encerrado.
```

Registro de execução - 02_cadastro_teclado

```
PS C:\programacao_software_basico_c\trabalho> .\produtos.exe
===== CONTROLE DE PRODUTOS =====
1. Cadastrar produto
2. Listar produtos
3. Buscar produto
4. Calcular valor do estoque
5. Atualizar quantidade
6. Sair
Escolha uma opcao: 1
=== Cadastro de produto ===
Codigo: 1002
Nome: Teclado mecanico
Categoria: Perifericos
Quantidade em estoque: 5
Preco unitario: R$ 25,50
Produto cadastrado com sucesso.
===== CONTROLE DE PRODUTOS =====
1. Cadastrar produto
2. Listar produtos
3. Buscar produto
4. Calcular valor do estoque
5. Atualizar quantidade
6. Sair
Escolha uma opcao: 6
Programa encerrado.
```

Registro de execução - 03_listar_produtos

```
PS C:\programacao_software_basico_c\trabalho> .\produtos.exe
===== CONTROLE DE PRODUTOS =====
1. Cadastrar produto
2. Listar produtos
3. Buscar produto
4. Calcular valor do estoque
5. Atualizar quantidade
6. Sair
Escolha uma opcao: 2
=== Produtos cadastrados ===
Codigo: 1001 | Nome: Mouse sem fio | Categoria: Perifericos | Quantidade: 2 | Preço: R$ 149.90
Codigo: 1002 | Nome: Teclado mecanico | Categoria: Perifericos | Quantidade: 5 | Preço: R$ 25.50
===== CONTROLE DE PRODUTOS =====
1. Cadastrar produto
2. Listar produtos
3. Buscar produto
4. Calcular valor do estoque
5. Atualizar quantidade
6. Sair
Escolha uma opcao: 6
Programa encerrado.
```

Registro de execução - 04_busca_encontrado

```
PS C:\programacao_software_basico_c\trabalho> .\produtos.exe
===== CONTROLE DE PRODUTOS =====
1. Cadastrar produto
2. Listar produtos
3. Buscar produto
4. Calcular valor do estoque
5. Atualizar quantidade
6. Sair
Escolha uma opcao: 3
=== Busca de produto ===
Codigo para buscar: 1001
Produto encontrado:
Codigo: 1001 | Nome: Mouse sem fio | Categoria: Perifericos | Quantidade: 2 | Preço: R$ 149.90
===== CONTROLE DE PRODUTOS =====
1. Cadastrar produto
2. Listar produtos
3. Buscar produto
4. Calcular valor do estoque
5. Atualizar quantidade
6. Sair
Escolha uma opcao: 6
Programa encerrado.
```

Registro de execução - 05_busca_nao_encontrado

```
PS C:\programacao_software_basico_c\trabalho> .\produtos.exe
===== CONTROLE DE PRODUTOS =====
1. Cadastrar produto
2. Listar produtos
3. Buscar produto
4. Calcular valor do estoque
5. Atualizar quantidade
6. Sair
Escolha uma opcao: 3
=== Busca de produto ===
Codigo para buscar: 9999
Produto com codigo 9999 nao encontrado.
===== CONTROLE DE PRODUTOS =====
1. Cadastrar produto
2. Listar produtos
3. Buscar produto
4. Calcular valor do estoque
5. Atualizar quantidade
6. Sair
Escolha uma opcao: 6
Programa encerrado.
```

Registro de execução - 06_total_antes_atualizacao

```
PS C:\programacao_software_basico_c\trabalho> .\produtos.exe
===== CONTROLE DE PRODUTOS =====
1. Cadastrar produto
2. Listar produtos
3. Buscar produto
4. Calcular valor do estoque
5. Atualizar quantidade
6. Sair
Escolha uma opcao: 4
=== Valor total do estoque ===
Valor total do estoque: R$ 427,30
===== CONTROLE DE PRODUTOS =====
1. Cadastrar produto
2. Listar produtos
3. Buscar produto
4. Calcular valor do estoque
5. Atualizar quantidade
6. Sair
Escolha uma opcao: 6
Programa encerrado.
```

Registro de execução - 07_atualizar_quantidade

```
PS C:\programacao_software_basico_c\trabalho> .\produtos.exe
===== CONTROLE DE PRODUTOS =====
1. Cadastrar produto
2. Listar produtos
3. Buscar produto
4. Calcular valor do estoque
5. Atualizar quantidade
6. Sair
Escolha uma opcao: 5
=== Atualizar quantidade em estoque ===
Codigo do produto: 1001
Nova quantidade: 4
Quantidade atualizada com sucesso.
===== CONTROLE DE PRODUTOS =====
1. Cadastrar produto
2. Listar produtos
3. Buscar produto
4. Calcular valor do estoque
5. Atualizar quantidade
6. Sair
Escolha uma opcao: 6
Programa encerrado.
```

Registro de execução - 08_listar_apos_atualizacao

```
PS C:\programacao_software_basico_c\trabalho> .\produtos.exe
===== CONTROLE DE PRODUTOS =====
1. Cadastrar produto
2. Listar produtos
3. Buscar produto
4. Calcular valor do estoque
5. Atualizar quantidade
6. Sair
Escolha uma opcao: 2
=== Produtos cadastrados ===
Codigo: 1001 | Nome: Mouse sem fio | Categoria: Perifericos | Quantidade: 4 | Preço: R$ 149.90
Codigo: 1002 | Nome: Teclado mecanico | Categoria: Perifericos | Quantidade: 5 | Preço: R$ 25.50
===== CONTROLE DE PRODUTOS =====
1. Cadastrar produto
2. Listar produtos
3. Buscar produto
4. Calcular valor do estoque
5. Atualizar quantidade
6. Sair
Escolha uma opcao: 6
Programa encerrado.
```

Registro de execução - 09_total_apos_atualizacao

```
PS C:\programacao_software_basico_c\trabalho> .\produtos.exe
===== CONTROLE DE PRODUTOS =====
1. Cadastrar produto
2. Listar produtos
3. Buscar produto
4. Calcular valor do estoque
5. Atualizar quantidade
6. Sair
Escolha uma opcao: 4
=== Valor total do estoque ===
Valor total do estoque: R$ 727.10
===== CONTROLE DE PRODUTOS =====
1. Cadastrar produto
2. Listar produtos
3. Buscar produto
4. Calcular valor do estoque
5. Atualizar quantidade
6. Sair
Escolha uma opcao: 6
Programa encerrado.
```

Registro de execução - 10_sair

```
PS C:\programacao_software_basico_c\trabalho> .\produtos.exe
===== CONTROLE DE PRODUTOS =====
1. Cadastrar produto
2. Listar produtos
3. Buscar produto
4. Calcular valor do estoque
5. Atualizar quantidade
6. Sair
Escolha uma opcao: 6
Programa encerrado.
```

Arquivo criado - produtos.txt

```
PS C:\programacao_software_basico_c\trabalho> Get-Content .\produtos.txt
1001|Mouse sem fio|Perifericos|4|149.90
1002|Teclado mecanico|Perifericos|5|25.50
```

Código-fonte completo

Arquivo: include/produto.h

```
#ifndef PRODUTO_H
#define PRODUTO_H

#define PRODUTO_TAMANHO_NOME 80
#define PRODUTO_TAMANHO_CATEGORIA 50

typedef struct {
    int codigo;
    char nome[PRODUTO_TAMANHO_NOME];
    char categoria[PRODUTO_TAMANHO_CATEGORIA];
    int quantidade;
    double preco;
} Produto;

#endif
```

Arquivo: include/arquivo_produtos.h

```
#ifndef ARQUIVO_PRODUTOS_H
#define ARQUIVO_PRODUTOS_H

#include <stddef.h>

#include "produto.h"

#define ARQUIVO_PRODUTOS_LIMITE 1000

typedef enum {
    ARQUIVO_PRODUTOS_OK,
    ARQUIVO_PRODUTOS_NAO_ENCONTRADO,
    ARQUIVO_PRODUTOS_DUPLICADO,
    ARQUIVO_PRODUTOS_ERRO,
    ARQUIVO_PRODUTOS_LIMITE_ATINGIDO
} ResultadoArquivoProdutos;

ResultadoArquivoProdutos arquivo_produtos_adicionar(const Produto *produto);
ResultadoArquivoProdutos arquivo_produtos_listar(
    Produto produtos[], size_t capacidade, size_t *quantidade);
ResultadoArquivoProdutos arquivo_produtos_buscar(int codigo, Produto *produto);
ResultadoArquivoProdutos arquivo_produtos_calcular_total(double *total);
ResultadoArquivoProdutos arquivo_produtos_atualizar_quantidade(int codigo, int quantidade);

#endif
```

Arquivo: include/entrada.h

```
#ifndef ENTRADA_H
#define ENTRADA_H

#include <stddef.h>

int entrada_ler_inteiro(const char *mensagem, int minimo, int *valor);
int entrada_ler_preco(const char *mensagem, double *valor);
int entrada_ler_texto(const char *mensagem, char *destino, size_t tamanho);

#endif
```

Arquivo: include/menu.h

```
#ifndef MENU_H
#define MENU_H

void menu_executar(void);

#endif
```

Arquivo: src/arquivo_produtos.c

```
#include "arquivo_produtos.h"

#include <errno.h>
#include <math.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define NOME_ARQUIVO "produtos.txt"
#define NOME_TEMPORARIO "produtos.tmp"
#define NOME_BACKUP "produtos.bak"
#define TAMANHO_LINHA 256
```

```

static int ler_produto(FILE *arquivo, Produto *produto) {
    char linha[TAMANHO_LINHA];
    char extra;
    int campos_lidos;

    if (fgets(linha, sizeof(linha), arquivo) == NULL) {
        return ferror(arquivo) ? -2 : 0;
    }
    if (strchr(linha, '\n') == NULL && !feof(arquivo)) {
        int caractere;
        while ((caractere = fgetc(arquivo)) != '\n' && caractere != EOF) {
        }
        return -1;
    }

    campos_lidos = sscanf(linha, "%d%79[^\n]|%49[^\n]|%d|%.1f %c",
        &produto->codigo, produto->nome, produto->categoria,
        &produto->quantidade, &produto->preco, &extra);
    if (campos_lidos != 5 || produto->codigo <= 0 || produto->quantidade < 0 ||
        !isfinite(produto->preco) || produto->preco < 0.0) {
        return -1;
    }
    return 1;
}

ResultadoArquivoProdutos arquivo_produtos_buscar(int codigo, Produto *produto) {
    FILE *arquivo = fopen(NOME_ARQUIVO, "r");
    Produto atual;
    int resultado_leitura;

    if (arquivo == NULL) {
        return errno == ENOENT ? ARQUIVO_PRODUTOS_NAO_ENCONTRADO : ARQUIVO_PRODUTOS_ERRO;
    }
    while ((resultado_leitura = ler_produto(arquivo, &atual)) != 0) {
        if (resultado_leitura == -2) {
            fclose(arquivo);
            return ARQUIVO_PRODUTOS_ERRO;
        }
        if (resultado_leitura > 0 && atual.codigo == codigo) {
            if (produto != NULL) {
                *produto = atual;
            }
            return fclose(arquivo) == 0 ? ARQUIVO_PRODUTOS_OK : ARQUIVO_PRODUTOS_ERRO;
        }
    }

    if (fclose(arquivo) != 0) {
        return ARQUIVO_PRODUTOS_ERRO;
    }
    return ARQUIVO_PRODUTOS_NAO_ENCONTRADO;
}

ResultadoArquivoProdutos arquivo_produtos_adicionar(const Produto *produto) {
    FILE *arquivo;
    ResultadoArquivoProdutos resultado_busca;
    int erro_escrita;
    int erro_fechamento;

    resultado_busca = arquivo_produtos_buscar(produto->codigo, NULL);
    if (resultado_busca == ARQUIVO_PRODUTOS_OK) {
        return ARQUIVO_PRODUTOS_DUPLICADO;
    }
    if (resultado_busca != ARQUIVO_PRODUTOS_NAO_ENCONTRADO) {
        return resultado_busca;
    }

    arquivo = fopen(NOME_ARQUIVO, "a");
    if (arquivo == NULL) {
        return ARQUIVO_PRODUTOS_ERRO;
    }
    erro_escrita = fprintf(arquivo, "%d|%.1f|%.2f\n", produto->codigo, produto->nome,
        produto->categoria, produto->quantidade, produto->preco) < 0;
    erro_fechamento = fclose(arquivo) != 0;
    return erro_escrita || erro_fechamento ? ARQUIVO_PRODUTOS_ERRO : ARQUIVO_PRODUTOS_OK;
}

ResultadoArquivoProdutos arquivo_produtos_listar(
    Produto produtos[], size_t capacidade, size_t *quantidade) {
    FILE *arquivo;
    Produto produto;
    int resultado_leitura;
    ResultadoArquivoProdutos resultado = ARQUIVO_PRODUTOS_OK;

    *quantidade = 0;
    arquivo = fopen(NOME_ARQUIVO, "r");
    if (arquivo == NULL) {
        return errno == ENOENT ? ARQUIVO_PRODUTOS_OK : ARQUIVO_PRODUTOS_ERRO;
    }

    while ((resultado_leitura = ler_produto(arquivo, &produto)) != 0) {

```

```

        if (resultado_leitura == -2) {
            resultado = ARQUIVO_PRODUTOS_ERRO;
            break;
        }
        if (resultado_leitura < 0) {
            continue;
        }
        if (*quantidade == capacidade) {
            resultado = ARQUIVO_PRODUTOS_LIMITE_ATINGIDO;
            break;
        }
        produtos[(*quantidade)++] = produto;
    }

    if (fclose(arquivo) != 0 && resultado == ARQUIVO_PRODUTOS_OK) {
        resultado = ARQUIVO_PRODUTOS_ERRO;
    }
    return resultado;
}

ResultadoArquivoProdutos arquivo_produtos_calcular_total(double *total) {
    FILE *arquivo;
    Produto produto;
    int resultado_leitura;

    *total = 0.0;
    arquivo = fopen(NOME_ARQUIVO, "r");
    if (arquivo == NULL) {
        return errno == ENOENT ? ARQUIVO_PRODUTOS_OK : ARQUIVO_PRODUTOS_ERRO;
    }

    while ((resultado_leitura = ler_produto(arquivo, &produto)) != 0) {
        if (resultado_leitura == -2) {
            fclose(arquivo);
            return ARQUIVO_PRODUTOS_ERRO;
        }
        if (resultado_leitura > 0) {
            *total += produto.quantidade * produto.preco;
        }
    }
    return fclose(arquivo) == 0 ? ARQUIVO_PRODUTOS_OK : ARQUIVO_PRODUTOS_ERRO;
}

ResultadoArquivoProdutos arquivo_produtos_atualizar_quantidade(int codigo, int quantidade) {
    FILE *original;
    FILE *temporario;
    Produto produto;
    int resultado_leitura;
    int encontrado = 0;
    int erro_leitura;
    int erro_fechamento_original;
    int erro_fechamento_temporario;

    original = fopen(NOME_ARQUIVO, "r");
    if (original == NULL) {
        return errno == ENOENT ? ARQUIVO_PRODUTOS_NAO_ENCONTRADO : ARQUIVO_PRODUTOS_ERRO;
    }
    temporario = fopen(NOME_TEMPORARIO, "w");
    if (temporario == NULL) {
        fclose(original);
        return ARQUIVO_PRODUTOS_ERRO;
    }

    while ((resultado_leitura = ler_produto(original, &produto)) != 0) {
        if (resultado_leitura < 0) {
            fclose(original);
            fclose(temporario);
            remove(NOME_TEMPORARIO);
            return ARQUIVO_PRODUTOS_ERRO;
        }
        if (produto.codigo == codigo) {
            produto.quantidade = quantidade;
            encontrado = 1;
        }
        if (fprintf(temporario, "%d|%s|%s|%d|%.2f\n", produto.codigo, produto.nome,
            produto.categoria, produto.quantidade, produto.preco) < 0) {
            fclose(original);
            fclose(temporario);
            remove(NOME_TEMPORARIO);
            return ARQUIVO_PRODUTOS_ERRO;
        }
    }

    erro_leitura = ferror(original);
    erro_fechamento_original = fclose(original) != 0;
    erro_fechamento_temporario = fclose(temporario) != 0;
    if (erro_leitura || erro_fechamento_original || erro_fechamento_temporario) {
        remove(NOME_TEMPORARIO);
        return ARQUIVO_PRODUTOS_ERRO;
    }
}

```

```

if (!encontrado) {
    remove(NOME_TEMPORARIO);
    return ARQUIVO_PRODUTOS_NAO_ENCONTRADO;
}

remove(NOME_BACKUP);
if (rename(NOME_ARQUIVO, NOME_BACKUP) != 0) {
    remove(NOME_TEMPORARIO);
    return ARQUIVO_PRODUTOS_ERRO;
}
if (rename(NOME_TEMPORARIO, NOME_ARQUIVO) != 0) {
    rename(NOME_BACKUP, NOME_ARQUIVO);
    remove(NOME_TEMPORARIO);
    return ARQUIVO_PRODUTOS_ERRO;
}
remove(NOME_BACKUP);
return ARQUIVO_PRODUTOS_OK;
}

```

Arquivo: src/entrada.c

```

#include "entrada.h"

#include <ctype.h>
#include <errno.h>
#include <limits.h>
#include <math.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

static int ler_linha(const char *mensagem, char *destino, size_t tamanho) {
    size_t comprimento;

    printf("%s", mensagem);
    if (fgets(destino, (int)tamanho, stdin) == NULL) {
        return 0;
    }

    comprimento = strlen(destino);
    if (comprimento > 0 && destino[comprimento - 1] == '\n') {
        destino[--comprimento] = '\0';
    } else {
        int caractere;
        while ((caractere = getchar()) != '\n' && caractere != EOF) {
        }
        printf("Entrada muito longa. Tente novamente.\n");
        return -1;
    }

    if (comprimento > 0 && destino[comprimento - 1] == '\r') {
        destino[comprimento - 1] = '\0';
    }
    return 1;
}

int entrada_ler_inteiro(const char *mensagem, int minimo, int *valor) {
    char linha[80];
    char *fim;
    long numero;
    int resultado;

    for (;;) {
        resultado = ler_linha(mensagem, linha, sizeof(linha));
        if (resultado == 0) {
            return 0;
        }
        if (resultado < 0 || linha[0] == '\0') {
            printf("Digite um numero inteiro valido.\n");
            continue;
        }

        errno = 0;
        numero = strtol(linha, &fim, 10);
        while (isspace((unsigned char)*fim)) {
            fim++;
        }
        if (errno == 0 && fim != linha && *fim == '\0' && numero >= minimo && numero <= INT_MAX) {
            *valor = (int)numero;
            return 1;
        }
        printf("Valor invalido. Digite um inteiro maior ou igual a %d.\n", minimo);
    }
}

int entrada_ler_preco(const char *mensagem, double *valor) {
    char linha[80];
    char *fim;
    size_t indice;

```

```

int resultado;
double numero;

for (;;) {
    resultado = ler_linha(mensagem, linha, sizeof(linha));
    if (resultado == 0) {
        return 0;
    }
    if (resultado < 0 || linha[0] == '\0') {
        printf("Digite um preco valido, por exemplo 129.90.\n");
        continue;
    }
    for (indice = 0; linha[indice] != '\0'; indice++) {
        if (linha[indice] == ',') {
            linha[indice] = '.';
        }
    }

    errno = 0;
    numero = strtod(linha, &fim);
    while (isspace((unsigned char)*fim)) {
        fim++;
    }
    if (errno == 0 && fim != linha && *fim == '\0' && isfinite(numero) && numero >= 0.0) {
        *valor = numero;
        return 1;
    }
    printf("Preco invalido. Use um numero maior ou igual a zero.\n");
}

int entrada_ler_texto(const char *mensagem, char *destino, size_t tamanho) {
    int resultado;

    for (;;) {
        resultado = ler_linha(mensagem, destino, tamanho);
        if (resultado == 0) {
            return 0;
        }
        if (resultado < 0) {
            continue;
        }
        if (destino[0] == '\0' || strchr(destino, '|') != NULL) {
            printf("O campo nao pode ficar vazio nem conter o caractere |.\n");
            continue;
        }
        return 1;
    }
}

```

Arquivo: src/menu.c

```

#include "menu.h"

#include "arquivo_produtos.h"
#include "entrada.h"

#include <stdio.h>

static void imprimir_produto(const Produto *produto) {
    printf("Codigo: %d | Nome: %s | Categoria: %s | Quantidade: %d | Preco: R$ %.2f\n",
        produto->codigo, produto->nome, produto->categoria,
        produto->quantidade, produto->preco);
}

static void cadastrar_produto(void) {
    Produto produto;
    ResultadoArquivoProdutos resultado;

    printf("\n=== Cadastro de produto ===\n");
    if (!entrada_ler_inteiro("Codigo: ", 1, &produto.codigo) ||
        !entrada_ler_texto("Nome: ", produto.nome, sizeof(produto.nome)) ||
        !entrada_ler_texto("Categoria: ", produto.categoria, sizeof(produto.categoria)) ||
        !entrada_ler_inteiro("Quantidade em estoque: ", 0, &produto.quantidade) ||
        !entrada_ler_preco("Preco unitario: R$ ", &produto.preco)) {
        printf("Cadastro cancelado.\n");
        return;
    }

    resultado = arquivo_produtos_adicionar(&produto);
    if (resultado == ARQUIVO_PRODUTOS_DUPLICADO) {
        printf("Ja existe um produto com esse codigo.\n");
    } else if (resultado == ARQUIVO_PRODUTOS_OK) {
        printf("Produto cadastrado com sucesso.\n");
    } else {
        printf("Nao foi possivel gravar o produto no arquivo.\n");
    }
}

static void listar_produtos(void) {

```

```

Produto produtos[ARQUIVO_PRODUTOS_LIMITE];
size_t quantidade;
size_t indice;
ResultadoArquivoProdutos resultado;

printf("\n=== Produtos cadastrados ===\n");
resultado = arquivo_produtos_listar(produtos, ARQUIVO_PRODUTOS_LIMITE, &quantidade);
if (resultado == ARQUIVO_PRODUTOS_ERRO) {
    printf("Erro durante a leitura do arquivo.\n");
    return;
}
if (resultado == ARQUIVO_PRODUTOS_LIMITE_ATINGIDO) {
    printf("O limite de produtos para listagem foi atingido.\n");
}
if (quantidade == 0) {
    printf("Nenhum produto cadastrado.\n");
    return;
}
for (indice = 0; indice < quantidade; indice++) {
    imprimir_produto(&produtos[indice]);
}
}

static void buscar_produto(void) {
    Produto produto;
    int codigo;
    ResultadoArquivoProdutos resultado;

    printf("\n=== Busca de produto ===\n");
    if (!entrada_ler_inteiro("Codigo para buscar: ", 1, &codigo)) {
        printf("Busca cancelada.\n");
        return;
    }

    resultado = arquivo_produtos_buscar(codigo, &produto);
    if (resultado == ARQUIVO_PRODUTOS_OK) {
        printf("Produto encontrado:\n");
        imprimir_produto(&produto);
    } else if (resultado == ARQUIVO_PRODUTOS_NAO_ENCONTRADO) {
        printf("Produto com codigo %d nao encontrado.\n", codigo);
    } else {
        printf("Erro durante a leitura do arquivo.\n");
    }
}

static void calcular_valor_estoque(void) {
    double total;

    printf("\n=== Valor total do estoque ===\n");
    if (arquivo_produtos_calcular_total(&total) != ARQUIVO_PRODUTOS_OK) {
        printf("Erro durante a leitura do arquivo.\n");
        return;
    }
    printf("Valor total do estoque: R$ %.2f\n", total);
}

static void atualizar_quantidade(void) {
    int codigo;
    int quantidade;
    ResultadoArquivoProdutos resultado;

    printf("\n=== Atualizar quantidade em estoque ===\n");
    if (!entrada_ler_inteiro("Codigo do produto: ", 1, &codigo) ||
        !entrada_ler_inteiro("Nova quantidade: ", 0, &quantidade)) {
        printf("Atualizacao cancelada.\n");
        return;
    }

    resultado = arquivo_produtos_atualizar_quantidade(codigo, quantidade);
    if (resultado == ARQUIVO_PRODUTOS_OK) {
        printf("Quantidade atualizada com sucesso.\n");
    } else if (resultado == ARQUIVO_PRODUTOS_NAO_ENCONTRADO) {
        printf("Produto com codigo %d nao encontrado.\n", codigo);
    } else {
        printf("Nao foi possivel atualizar o arquivo de produtos.\n");
    }
}

static void exibir_menu(void) {
    printf("\n===== CONTROLE DE PRODUTOS =====\n");
    printf("1. Cadastrar produto\n");
    printf("2. Listar produtos\n");
    printf("3. Buscar produto\n");
    printf("4. Calcular valor do estoque\n");
    printf("5. Atualizar quantidade\n");
    printf("6. Sair\n");
}

void menu_executar(void) {
    int opcao;

```

```

for (;;) {
    exibir_menu();
    if (!entrada_ler_inteiro("Escolha uma opcao: ", 1, &opcao)) {
        printf("\nPrograma encerrado.\n");
        return;
    }
    switch (opcao) {
        case 1:
            cadastrar_produto();
            break;
        case 2:
            listar_produtos();
            break;
        case 3:
            buscar_produto();
            break;
        case 4:
            calcular_valor_estoque();
            break;
        case 5:
            atualizar_quantidade();
            break;
        case 6:
            printf("Programa encerrado.\n");
            return;
        default:
            printf("Opcao invalida. Escolha uma opcao de 1 a 6.\n");
    }
}
}

```

Arquivo: src/main.c

```

#include "menu.h"

int main(void) {
    menu_executar();
    return 0;
}

```